DISASTER RESOURCE ALLOCATION SIMULATOR

Kabilan A S - 3122255001071
Kavinkumar L - 3122255001075
Nitin Karthikeyan - 3122245001705

SSN College of Engineering
Department of Computer Science and Engineering
Academic Year: 2026-2027

CERTIFICATE

This is to certify that the project report entitled DISASTER RESOURCE ALLOCATION SIMULATOR is the bona fide work of Kabilan A S, Kavinkumar L, and Nitin Karthikeyan who carried out the project work under my supervision.

DECLARATION

We hereby declare that the project entitled DISASTER RESOURCE ALLOCATION SIMULATOR is a record of an original work.

ACKNOWLEDGEMENT

We thank our faculty mentors and the CSE Department at SSN College of Engineering.

ABSTRACT

The Disaster Resource Allocation Simulator is an advanced software system designed to model, calculate, and distribute critical resources (Water, Food, Medicine, Fuel, and Rescue Teams). Using a Java-based backend running a custom multithreaded simulation engine and a modern React/Vite frontend, the system processes demographic and geographical data to calculate requirements dynamically. The engine uses priority-based allocation by computing zone criticality (Population * Severity * Medical Need) and demonstrates core OOP principles and concurrent execution using wait/notify thread synchronization. The system exposes a RESTful API and supports what-if analysis.

CHAPTER 1 - INTRODUCTION

1.1 Background: Efficient allocation of resources during natural disasters is critical.

1.2 Problem Statement: Manual resource allocation is slow and error-prone.

1.3 Motivation: Provide a robust tool to simulate disaster response.

1.4 Objectives: Priority-based algorithm, concurrent engine, React frontend.

1.5 Scope: Covers Floods, Earthquakes, Cyclones, Wildfires. Tracks resources and teams.

CHAPTER 2 - EXISTING AND PROPOSED SYSTEM

Existing System: Traditional systems rely on manual spreadsheets. Proposed System: Dynamically calculates resource needs using Java multithreading to simulate independent rescue teams.

CHAPTER 3 - REQUIREMENTS

Functional: Create scenarios, allocate resources dynamically, run concurrent simulation, monitor status.
Non-Functional: Modularity, Reliability, Responsiveness.
Software: Java 17/21, JDK HttpServer, React, Vite, TS.

CHAPTER 4 - SYSTEM ARCHITECTURE

React Frontend <-> Java HTTP API <-> Simulation Engine <-> File Data Store

Figure 4.1 - System Architecture Diagram

CHAPTER 5 - COMPLETE SYSTEM WORKFLOW

START -> React App -> Init DataStore -> Setup Disaster -> Load Zones -> Calc Priority -> Allocate Resources -> Run RescueTasks -> Display Results -> END

CHAPTER 6 - SYSTEM DESIGN

Disaster (Abstract) <|-- Flood, Earthquake
Resource (Abstract) <|-- Consumable <|-- Water, Food, Medicine, Fuel
RescueTask implements Runnable

CHAPTER 7 - OBJECT ORIENTED PROGRAMMING

7.1 Encapsulation: Zone.java fields.
7.2 Inheritance: Disaster & Resource.
7.3 Polymorphism: overridden drain() method.
7.6 Interfaces: DataStore.
7.8 Collections: List<Zone>, Map<String, Resource>.
7.11 Exception Handling: InsufficientResourceException.

CHAPTER 8 - DISASTER AND ZONE MODEL

priorityScore = population * severity * medicalNeed
demandPercent = 100 + baseImpact + typeBonus

CHAPTER 9 - RESOURCE MANAGEMENT AND ALLOCATION

Water: pop * 3 * hours * demandPercent / 2400
Food: pop * 2 * hours * demandPercent / 2400
Medicine: pop * medicalNeed * demandPercent / 10000
Fuel: distance * 10
Teams: ceil(population / 2500)

CHAPTER 10 - SIMULATION ENGINE

Tracks states: CONFIGURED, RUNNING, COMPLETED. Triggers parallel RescueTask threads.

CHAPTER 11 - MULTITHREADING AND SYNCHRONIZATION

RescueTask implements Runnable. TeamPool uses synchronized acquire() with wait() and release() with notifyAll() to prevent race conditions. Thread states: NEW -> RUNNABLE -> WAITING -> TERMINATED.

CHAPTER 12 - FILE HANDLING

FileManager parses flat text files: zones.txt, resources.txt, disasters.txt. Results saved as JSON strings.

CHAPTER 13 - HTTP API

Endpoints: POST /api/simulation/start, GET /api/simulation/status, GET /api/simulation/results, GET /api/zones, GET /api/resources, POST /api/simulation/what-if, GET /api/history.

CHAPTER 14 - REACT FRONTEND

Built with React and Vite. Components: EmergencyCommandCenter, ResourceAllocation, SimulationControl. (Insert Screenshots here).

CHAPTER 15 - FRONTEND / BACKEND INTEGRATION

Communicates via fetch API in simulationApi.ts. Uses polling for live status.

CHAPTER 16 - TESTING

9 tests run using JUnit. 2 PASS, 7 FAIL. Failures primarily due to method signature changes (e.g. Flood constructor and calculatePriority).

CHAPTER 17 - GOLDEN SCENARIO

Verified values for Flood, Severity 5, Duration 24, Population 50,000:
Water Req: 180,000, Shortage: 60,000
Food Req: 120,000, Shortage: 0
Medicine Req: 1,848, Shortage: 348
Fuel Req: 1,150, Shortage: 150
Teams Req: 22, Shortage: 12

CHAPTER 18 - RESULTS AND DISCUSSION

Algorithm dynamically prioritizes Zone A. Concurrent tasks execute safely.

CHAPTER 19 - WHAT-IF ANALYSIS

Allows dynamic temporary overrides of severity/population without mutating DataStore.

CHAPTER 20 - LIMITATIONS

Text-file persistence, lack of real GIS integration, rule-based optimization.

CHAPTER 21 - FUTURE ENHANCEMENTS

Database integration, Cloud deployment, Map integrations.

CHAPTER 22 - CONCLUSION

Successfully demonstrates complex logic, OOP, and multithreading in Java, unified with a React frontend.